Stacks and Queues

The stack API and an array stack

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026

By the end you can

1. tell a data type from a data structure
2. describe a stack (LIFO) and a queue (FIFO) and trace both by hand
3. state the stack API and the three performance specifications
4. implement a stack of strings with an array
5. explain why the array stack does not meet the specifications

The call stack you have already overflowed

The recursive `sum(20000)` crashed. With which error – and what was full?
→ `StackOverflowError`: the call stack was full.

- Every call puts a frame on **top** of the stack
- Every return takes the **top** frame off
- The method that returns first is the one that was called last

Today: the same idea as a data type of our own – the **stack** – and its twin, the **queue**

Other stacks: the back button of a browser, undo in an editor, evaluating `(1 + ((2 + 3) * (4 * 5)))`

companion card 1: `birkenkrahe.github.io/demos/array-stack.html`

Data types and data structures

Data type

- set of values
- set of operations on those values
- built in: `int`, `double`, `String`
- our own: `Counter`, `Stopwatch`, ...

Data structure

- represents data in memory
- and the relationships among the data
- built in: arrays
- not built in: linked lists, trees

described by an **API**

Design question for every data type: which data structure?

- Memory: how much is needed?
- Time: how long do the data type's methods take?

Video CS.12.A, slide 3

Two collections: stack and queue

A **collection** holds a group of items of the same type: add, remove, is it empty?, size

Stack – remove the item **most** recently added
LIFO: last in, first out
push, pop
a pile of mail, newest on top

Queue – remove the item **least** recently added
FIFO: first in, first out
enqueue, dequeue
the line at a ticket counter

Key characteristic: no limit on the size of the collection.
The client never says how many items there will be.
So client code works unchanged as the data grows – with a fixed size, it would have to be edited each time.

Book §4.3, pp. 566-567; video CS.12.A, slide 4

Question: the same input on a stack and a queue

One operation per word: - means remove and print, any other word means add

```
to be or not to - be - - that - - - is
```

1. What does a **stack** print?
2. What does a **queue** print?
3. What is left at the end?

Answer

```
to be or not to - be - - that - - - is
```

	prints	**left at the end**
stack	`to be not that or be`	`is to` (top first)
queue	`to be or not to be`	`that is` (first first)

- The queue gives the words back in the order they came in, wherever the - are
- The stack reverses runs of pushes

Video CS.12.A, slides 5-6 ·

companion card 2: `birkenkrahe.github.io/demos/array-stack.html`

The stack API

`public class Stack<Item>`	
`Stack<Item>()`	create an empty stack
`void push(Item item)`	add an item
`Item pop()`	remove and return the item most recently added
`boolean isEmpty()`	is the stack empty?
`int size()`	number of items on the stack

- `Item` is a type parameter, as in `Comparable<Counter>`: the client chooses the type, e.g. `Stack<String>`
- The client calls `isEmpty()` before `pop()`: popping an empty stack is the client's mistake
- More than one implementation of an API – a client works with any of them
- Today a simpler version: `ArrayStackOfStrings`, strings only

Video CS.12.A, slide 7 · book §4.3, p. 568 (strings only)

Question: abstract data type or data structure?

A client uses `Stack<String>` only through `push()`, `pop()` and `isEmpty()`. The stack keeps its items in an array, with the top item at the end of the array.

1. Does the client code change if the top item moves to `items[0]`?
2. Which is a **data type**, an **abstract data type**, a **data structure** or an **implementation**?

   `int` · `Stopwatch` · a stack · a queue · an array · a stack that keeps its items in an array

Answer

1. **No.** The client sees only the API; how the items are kept is hidden. A data type whose representation is hidden from the client is an **abstract data type** (ADT).
2. Four terms:

Term	What it is	Examples
data type	values + operations on them	`int`, `double`
abstract data type	a data type whose representation is hidden: the client sees only the API	`Stopwatch`, stack, queue
data structure	how data and their relationships are represented in memory	array
implementation	an ADT built on a data structure	a stack on an array

Every ADT is a data type. Our first stack ADT runs on an array.

Video CS.12.A, slides 3-4

Performance specifications

An API says **what** the methods do. For stacks and queues we also require **how fast** and **how big**:

1. All operations take **constant time**
2. Memory use is **linear in the number of items** (when the collection is not empty)
3. **No limit** within the code on the size of the collection

Why in the API?

- The client sees only the API: a slow `push()` hidden in an implementation is a bug the client cannot find
- Countless programs use stacks and queues: a slow one slows them all

Video CS.12.A, slide 8

Check: stacks, queues and their API

True or false?

1. A queue removes the item least recently added → true
2. Both have `isEmpty()` and `size()` operations → true
3. The two APIs differ only in which item is removed and returned → true
4. After `push(a)`, `push(b)`, `pop()`, `push(c)`, `pop()`, the stack holds b → false: it holds a
5. A stack has a fixed maximum number of items in its API → false: the API sets no limit

Which item does `pop()` remove and return?

→ The item added most recently (LIFO).

Why does the API set no limit on the number of items?

→ So client code keeps working as the data grows.

An array stack: two decisions

Two simplifications of the `Stack<Item>` API:

1. strings only
2. the client gives a **capacity** to the constructor – so an array can hold the items

Where is the top?

- `items[0]` is the bottom, `items[n-1]` is the top
- push and pop work at the **end** of the array: nothing moves
- n counts the items – and is the index of the next free slot

Why not the top at `items[0]`?

→ Every push and pop would shift all other items: linear time, not constant.

Video CS.12.C, slides 22-23 (“strawman” stack)

ArrayStackOfStrings

→ **Switch to editor · code-along**

1. Get the test data **`tobe.txt`**
2. Fields **`String[] items`**, **`int n`**, and the constructor with a capacity
3. **`isEmpty()`**, **`size()`**, **`push()`**, **`pop()`**: one line each
4. Test client: `-` pops and prints, any other word is pushed
5. Run it with capacity 5 on `tobe.txt`

data: `birkenkrahe.github.io/data/tobe.txt` · classpath: `.:stdlib.jar` · book Program 4.3.1, p. 570

n++ and --n

```
private String[] items;
private int n = 0;

public boolean isEmpty()
{ return n == 0; }

public int size()
{ return n; }

public void push(String item)
{ items[n++] = item; }

public String pop()
{ return items[--n]; }
```

- `items[n++] = item;` stores into `items[n]`, **then** increments `n`: the new item goes into the free slot
- `return items[--n];` decrements `n` **first**, then returns `items[n]`: the top is at `n-1`

What goes wrong with `return items[n--];`?

→ It returns the free slot above the top: `null` or an old item – and on a full stack an `ArrayIndexOutOfBoundsException`.